

MID-STATE TECHNICAL COLLEGE • IT PROGRAMS

Efficient Deployment Strategies

Unit 6 • Virtualization and Software Defined Data Centers (10-150-130)



Learning Outcomes

By the end of this unit, you will be able to:



Build and maintain templates

Prepare a generalized golden image, convert it to a template, and version it over time.



Choose the right clone type

Select full or linked clones based on storage backend, workload, and independence needs.



Automate deployments

Use cloud-init, scripts, and infrastructure as code to provision guests repeatably.



Plan mass deployments

Size, stage, and roll out many instances without overwhelming storage or the network.

LESSON 1

Templates and Golden Images



Why Template-Based Deployment?

- A **template** is a read-only master guest that exists only to be cloned. It is never started directly.
- A **golden image** is the tested, patched, hardened build that you turn into a template.
- Cloning replaces a full OS install with a copy operation, turning minutes or hours into seconds.
- Every clone starts from the same baseline, which removes **configuration drift** between servers.



Templates in Proxmox VE

Once converted, a template's icon changes and the **Start** button disappears.

You can still edit its hardware, but changing the OS means cloning it, updating, and converting again.

Preparing the Golden Image

Bake it in

- OS fully patched at build time
- Guest agent: `qemu-guest-agent` or VirtIO tools
- `cloud-init` (Linux) or Cloudbase-Init (Windows)
- Security baseline, logging, and monitoring agents

Strip it out

- SSH host keys and the `/etc/machine-id` value
- User accounts, passwords, and SSH authorized keys
- Logs, shell history, and application data
- Hard-coded IPs and MAC-bound network configs

Rule of thumb: if two clones must never share it, it does not belong in the template. Let cloud-init or sysprep generate it on first boot.

Generalizing the Guest

Run these inside a Linux golden image as the very last step, then shut down:

```
sudo apt update && sudo apt -y full-upgrade
sudo cloud-init clean --logs
sudo truncate -s 0 /etc/machine-id
sudo rm -f /etc/ssh/ssh_host_*
history -c && sudo poweroff
```

- New SSH host keys and a new `machine-id` are generated on each clone's first boot.
- Don't log back in after cleaning. Doing so recreates what you just removed.

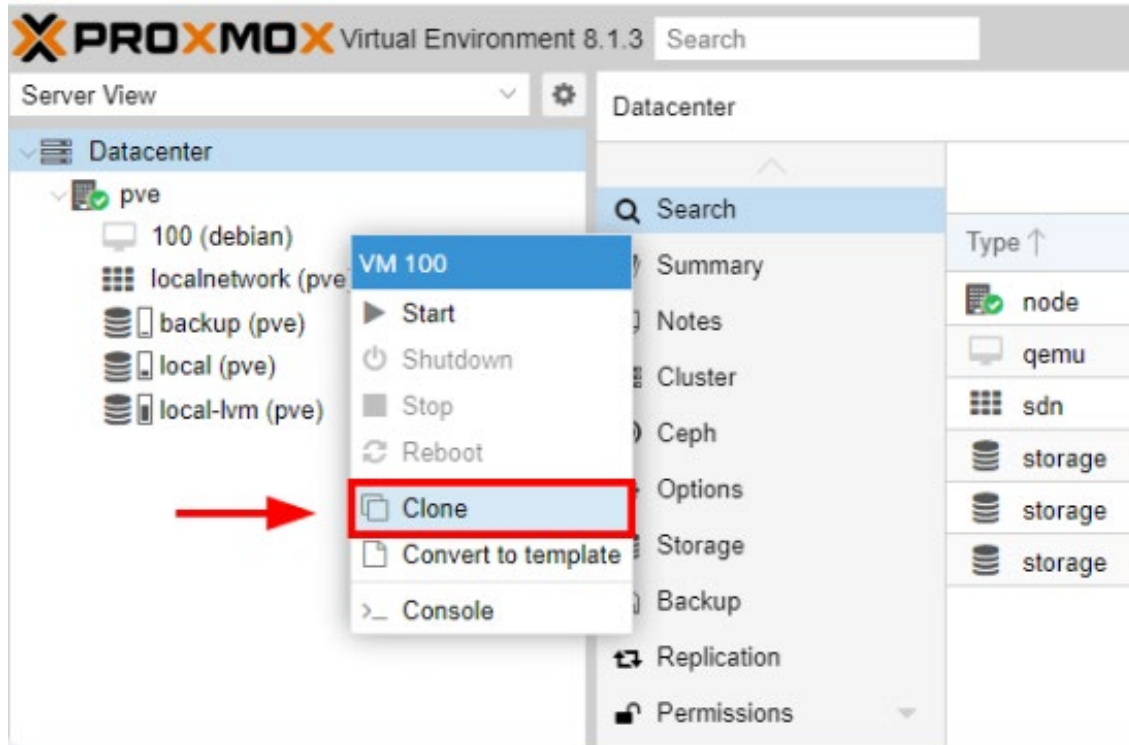


Windows: sysprep

`sysprep /generalize /oobe /shutdown` removes the machine SID and hardware-specific state.

Pair it with an answer file so clones finish setup unattended.

Converting to a Template



- Shut the VM down, then right-click and choose **Convert to template**.
- From the shell: `qm template 9000` or `pct template 9001`.
- Conversion is one-way. To change the OS later, full-clone it, update, and convert again.
- Reserve an ID range, such as `9000+`, so templates sort together.

Container Templates

Containers have three template sources, and each one fits a different job:

Source	How you get it	Best for
Appliance templates	<code>pveam update</code> then <code>pveam download</code>	Clean distro bases: Debian, Ubuntu, Alpine, Rocky
OCI images (9.1+)	Pull from a registry such as Docker Hub	Packaged apps without building Docker inside a VM
Your own CT template	Build a CT, then <code>pct template <id></code>	A hardened base with your org's tooling preinstalled

OCI application containers remain a technology preview in Proxmox VE 9.x. Use system containers for production workloads.

Template Lifecycle and Versioning

Treat templates like software releases. Version them, test them, and retire them deliberately.

Stage	What happens	Example
Build	Patch and harden a full clone of the current template	VM 9010 from <code>tmpl-deb13-2026.09</code>
Test	Clone it, boot it, and confirm cloud-init and agents work	Scratch VM, then destroy it
Release	Convert, name it with a version, and add a note	<code>tmpl-deb13-2026.10</code>
Retire	Remove the old version once nothing links to it	Keep N-1 for rollback

Monthly cadence: rebuilding templates on patch day keeps new clones current from the first boot.

LESSON 2

Full and Linked Clones



Two Ways to Clone

Full clone

- Complete, independent copy of every disk
- Uses as much space as the source
- Takes time to copy, depending on disk size and storage speed
- Survives deletion of the template

Linked clone

- Shares the template's disk and stores only its changes
- Uses almost no space at creation
- Ready in seconds, regardless of disk size
- Cannot run without the base template

Dependency: Proxmox blocks deleting a template while linked clones still use it. Losing the base disk would break every clone at once.

Comparing Deployment Methods

Method	Storage use	Independence	Best use case	Speed
Full clone	Full copy of each disk	Fully independent	Production servers	Minutes
Linked clone	Changes only (CoW)	Needs base template	Labs, dev, and test	Seconds
Container clone	Small root filesystem	Full or linked	Lightweight services	Seconds
Template + cloud-init	Template + each clone	Per-clone settings	Standardized fleets	Seconds to minutes

Times assume local SSD or NVMe storage. Full clones over a 1 GbE storage network can take much longer.

How Linked Clones Work: Copy-on-Write

- **Copy-on-write (CoW)**: unchanged blocks are read from the template. Changed blocks are written to the clone's own layer.
- On file storage, the clone is a `qcow2` file whose **backing file** is the template disk.
- On ZFS, LVM-thin, and Ceph RBD, the storage clones a snapshot of the template volume.
- Linked clones grow as data changes. A heavily rewritten clone can end up as large as a full copy.



Performance trade-off

Many clones reading the same base blocks create an I/O hot spot on the template's disk.

Full clones give more predictable I/O, so production usually gets full clones.

Storage Decides What Is Possible

Linked clones need storage that supports snapshots or backing files:

Storage type	Linked clones	Notes
NFS / directory (qcow2)	Yes	Our <code>VMLab-NFS</code> share works for linked clones
ZFS, LVM-thin, Ceph RBD	Yes	Native snapshot-based clones
LVM (thick) on iSCSI	No	9.x volume-chain snapshots are a tech preview, with no linked clones
Raw iSCSI LUN	No	Full clones only

Watch for preallocation: a directory store set to `preallocation falloc` makes linked clones allocate full size. Use `metadata` or the default.

Cloning from the Command Line

```
# Linked clone from template 9000 (template source required)
qm clone 9000 201 --name web01 --full 0
# Full clone onto a specific storage, throttled to 100 MiB/s
qm clone 9000 202 --name db01 --full 1 --storage local-zfs --bwlimit 102400
# Container clone from CT template 9001
pct clone 9001 301 --hostname app01
```

- `qemu-img info` on a linked clone's disk shows its **backing file**, the template.
- Each clone gets new MAC addresses automatically. The guest OS identity is up to your template prep.

LESSON 3

Deployment Automation



Declarative vs. Imperative

	Imperative	Declarative
You describe	The steps to perform, in order	The end state you want
Run it twice	May create duplicates or fail	No change: it is idempotent
Examples	qm / pct scripts, PowerShell	Terraform / OpenTofu, Ansible
Strength	Fast to write for one-off tasks	Version-controlled, repeatable, detects drift

In practice: declare the infrastructure, then use imperative scripts for the one-off steps that do not fit a model.

Cloud-Init: Per-Clone Customization

Proxmox attaches a small cloud-init drive. The guest reads it on first boot:

```
qm set 9000 --ide2 local-lvm:cloudinit
qm set 9000 --ciuser labadmin --sshkeys ~/.ssh/id_ed25519.pub
qm set 202 --ipconfig0 ip=192.168.50.21/24,gw=192.168.50.1
qm set 202 --nameserver 10.22.2.254 --searchdomain mstclab.local
qm cloudinit dump 202 user
```

- Settings on the template become defaults. Settings on a clone override them.
- For advanced setup, `--cicustom` points to a snippet on storage with the `snippets` content type.

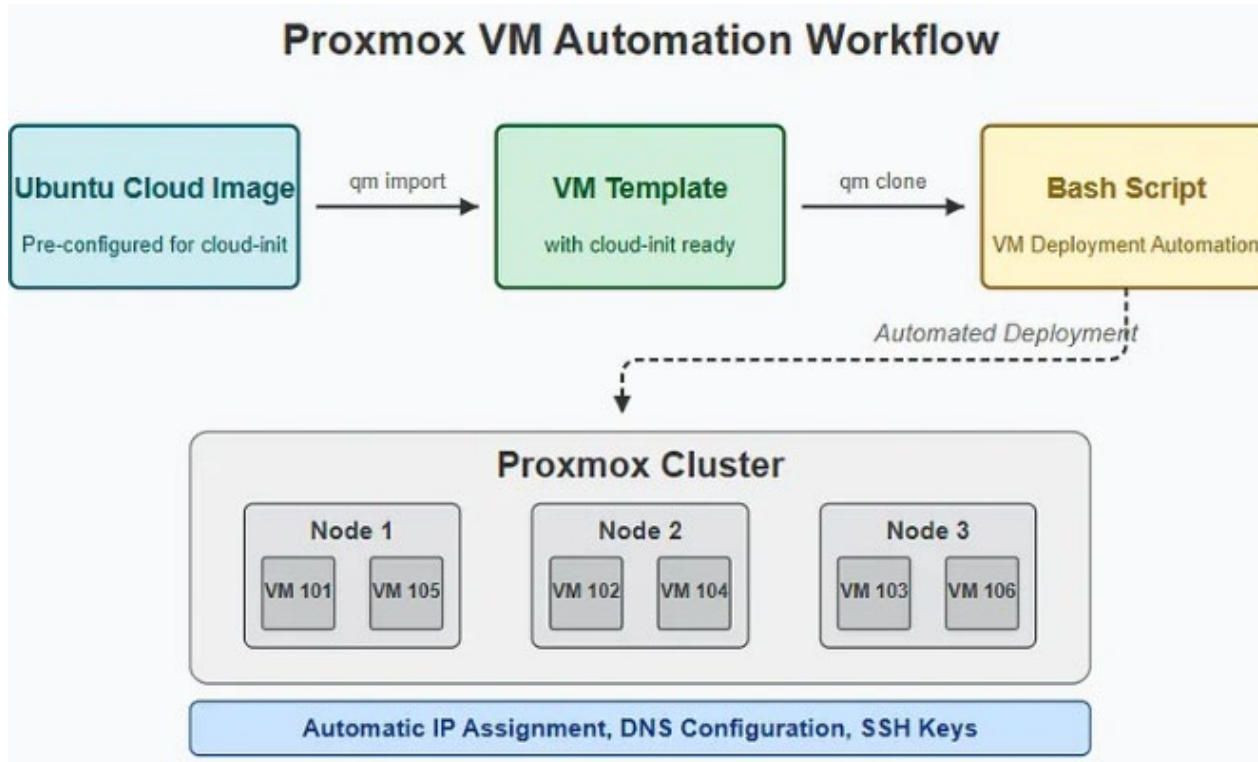


What it sets

User, password or SSH keys, hostname, IP or DHCP, DNS, and package upgrades on first boot.

Change a value, then click **Regenerate Image** before booting.

A Cloud-Image Automation Workflow



- Download a vendor **cloud image** (Debian `genericcloud`, Ubuntu cloud image).
- Import the disk with `qm disk import` and attach a cloud-init drive.
- Convert it to a template once.
- Clone and customize as many VMs as you need with a script or IaC tool.

Scripted Deployment

A few lines of bash turn one template into five configured web servers:

```
for i in 1 2 3 4 5; do
  id=$((210 + i))
  qm clone 9000 $id --name web0$i --full 0
  qm set $id --ipconfig0 ip=192.168.50.$((20 + i))/24,gw=192.168.50.1
  qm start $id
done
```

- Every GUI action is a REST call. `pvesh` and the API let any language do the same thing.
- Run it twice and `qm clone` fails because the IDs exist. Scripts must check state themselves.

Infrastructure as Code: Terraform and OpenTofu

```
resource "proxmox_virtual_environment_vm" "web" {  
  count      = 3  
  name       = "web0${count.index + 1}"  
  node_name  = "WRPOD01-Host1"  
  clone { vm_id = 9000 }  
  initialization {  
    ip_config { ipv4 { address = "dhcp" } }  
  }  
}
```

- `tofu plan` previews changes. `tofu apply` makes them. Change `count` to scale.
- The code lives in Git, so every change is reviewed, tracked, and reversible.



Which provider?

`bpg/proxmox` is the actively maintained provider for Terraform and OpenTofu.

The older `Telmate/proxmox` still fills search results but lags current Proxmox releases.

Pin the version and commit the lock file.

Configuration Management: Ansible

```
- name: Configure web servers
  hosts: web
  become: true
  tasks:
    - name: Install nginx
      ansible.builtin.apt:
        name: nginx
        state: present
    - name: Start and enable nginx
      ansible.builtin.service:
        name: nginx
        state: started
        enabled: true
```

- `ansible-playbook -i inventory.ini web.yml` runs it against every host in the `web` group.
- A second run reports **ok**, not **changed**. The playbook is **idempotent**.



Why Ansible fits here

Agentless: it connects over SSH with the key cloud-init placed, so nothing is added to the template.

A **playbook** in YAML describes the desired state of each server.

`community.proxmox` can also create guests and build its inventory from the Proxmox API.

Choosing the Right Tool for Each Layer

Real pipelines chain tools together. Each one owns a different layer:

Layer	Tool	Job
Provision	Terraform / OpenTofu	Create VMs, disks, and networks from templates
First boot	cloud-init	Set identity, users, keys, and IP on each clone
Configure	Ansible (<code>community.proxmox</code>)	Install apps and enforce settings over SSH
Orchestrate	CI/CD pipeline	Run the steps above on each Git commit

Proxmox modules moved from `community.general` to `community.proxmox`. Older playbooks show deprecation warnings.

Securing the Automation

- Give automation its own **API token** with **privilege separation** and only the roles it needs.
- Never commit tokens, passwords, or private keys to Git. Use environment variables or a secrets manager.
- Scope permissions to a pool or path, such as `/pool/automation`, rather than `/`.
- Review the Proxmox task log and API audit trail after each automated run.



SECURITY FOCUS

An automation credential can create, delete, or copy every guest it can see. Treat it like a root password.

Why it matters: A token pushed to a public repo can be found and abused within minutes.

LESSON 4

Mass Deployment Strategies



Scaling Up vs. Scaling Out

Vertical: scale up

- Add vCPU and RAM to one bigger instance
- Simple, but limited by the host's hardware
- The single instance is a single point of failure

Horizontal: scale out

- Add more identical instances behind a load balancer
- Limited mainly by cluster capacity
- One instance can fail without an outage

Same idea in the cloud: Azure scale sets, AWS Auto Scaling groups, and Kubernetes ReplicaSets all clone from one image to add or remove instances on demand.

Deployment Storms and Bottlenecks

- A **deployment storm** is many clones, boots, or updates at once, all competing for the same resources.
- **Storage I/O** saturates first. Full clones copy every block, and boot storms hit the same blocks.
- Deploy time grows roughly linearly until queues build, then it rises sharply.
- Stagger batches, cap bandwidth with `--bwlimit`, and prefer linked clones for short-lived fleets.



Watch these metrics

Storage latency and IO delay on the node Summary page.

Network throughput on storage and migration links.

The task queue in the bottom panel.

Rollout Patterns

Never push a change to every instance at once. Pick a pattern that limits the blast radius:

Pattern	How it works	Rollback
Canary	Deploy to a small share first, watch, then expand	Stop the rollout and revert the few canaries
Rolling	Replace instances in batches across the fleet	Roll the remaining batches back to N-1
Blue-green	Build a full parallel set, then switch traffic	Switch traffic back. It's instant but doubles capacity

Pipeline environments: promote every template and change through at least dev, test, and production before it reaches users.

Planning a Mass Deployment

- **Capacity:** template size times clone count for full clones, or the expected change rate for linked clones.
- **Sequencing:** map dependencies so DNS and databases come up before the apps that need them.
- **Batching:** size each wave to what storage and network can absorb, then monitor and adjust.
- **Failure handling:** at scale, something will fail. Plan retries, health checks, and rollback in advance.



Quick estimate

40 full clones of a 32 GB template need about 1.3 TB.

40 linked clones may start under 1 GB and grow with use.

Thin provisioning hides this until the pool fills.

Security Across the Deployment Pipeline



SECURITY FOCUS

Templates **multiply** everything in them. A secret, weak password, or unpatched package in the golden image ends up in every clone.

Generalize every template, rebuild it on a regular patch cadence, and give automation the narrowest API token that works.

Keep IaC and cloud-init files in version control, with no secrets inside.

Why it matters: One mistake in a template is deployed hundreds of times before anyone notices.

Builds on Unit 3: the API tokens, roles, and privilege separation you configured there are what keep automation safe here.

Hands-On Lab: Templates and Clones

Physical POD lab · Proxmox VE on Host1 · details and due date in Blackboard Ultra

- Prepare, generalize, and convert a Linux golden image to a template.
- Create a full clone and a linked clone, then compare their disk usage.
- Build a cloud-init template from a vendor cloud image and deploy a customized clone.
- Script the deployment of several clones and verify each one boots with a unique identity.



Graded on the live system: the template, both clone types, and an SSH-reachable cloud-init clone.

Key Takeaways

-  A generalized, versioned template is the starting point for every efficient deployment.
-  Full clones are independent. Linked clones are fast and small but depend on the template and on storage support.
-  Cloud-init, scripts, and IaC turn one template into many consistent, repeatable deployments.
-  Mass deployment is about pacing: stagger, monitor, and roll out in controlled waves.
-  Templates multiply mistakes. Keep them clean, patched, and free of secrets.

MID-STATE TECHNICAL COLLEGE · IT PROGRAMS

Questions?

Unit 6 · Efficient Deployment Strategies